



《推荐系统算法实践》 (SK3016)

第十五章 推荐系统架构（一）

查安秦

软件与人工智能学院

E-mail: zaq@mail.seig.com.cn



1. 整体框架

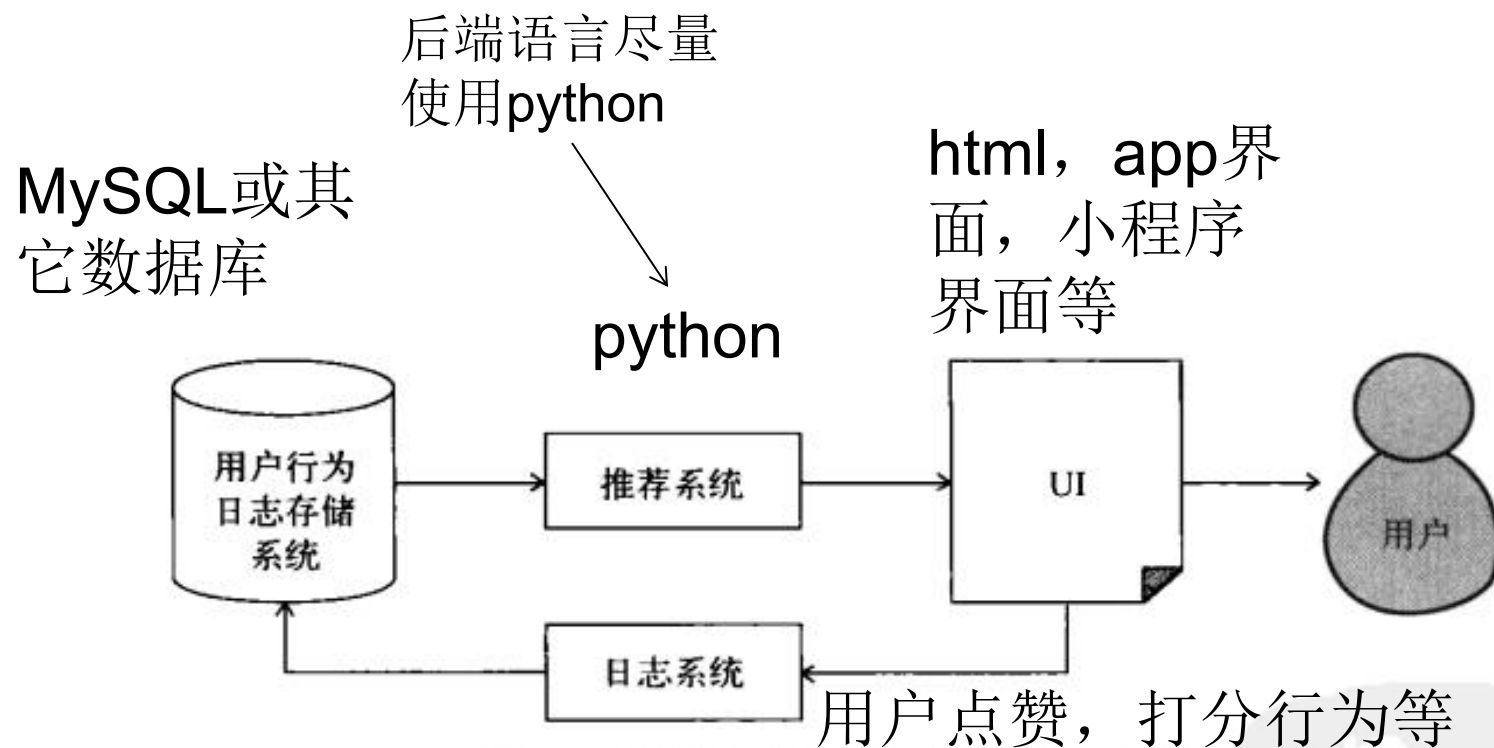


图7-1 推荐系统和其他系统之间的关系



2. PyCharm环境搭建

Two-factor authentication is available!

To enable an extra layer of security for your JetBrains Account, turn on two-factor auth.



Buy new license

1 Educational Pack License



JetBrains Educational Pack

Download

License ID:

Licensed to: 安秦查

License restriction: For educational use only

Valid through: December 12, 2025

Following products included:

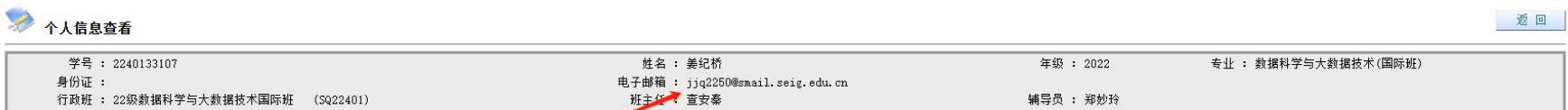
- Aqua
- CLion
- DataGrip
- DataSpell
- dotCover
- dotMemory
- dotTrace
- GoLand
- IntelliJ IDEA Ultimate
- PhpStorm
- PyCharm
- ReSharper
- ReSharper C++
- Rider
- RubyMine
- RustRover
- WebStorm

After downloading and installing the software, simply run it and follow the on-screen prompts to sign in with your JetBrains Account.

确认PyCharm使用的是Enterprise版本，Community版本理论上也可行



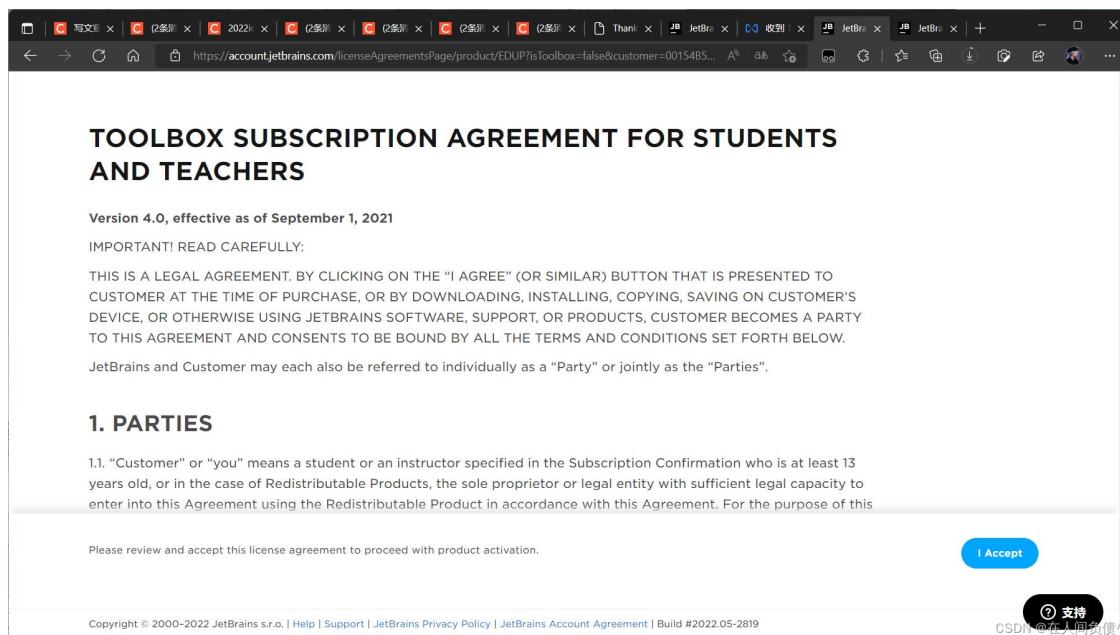
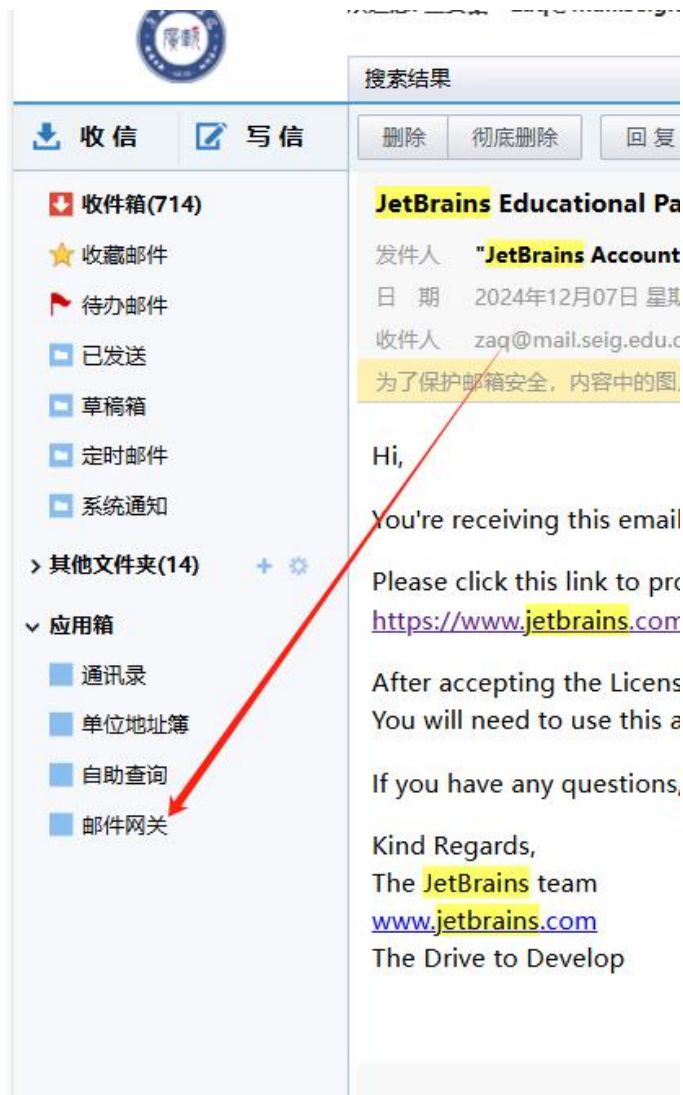
2. PyCharm环境搭建



备注：《课程名称》是红颜色的表示不及格 《修读学年学期》是空白的表示没选过这门课



2. PyCharm环境搭建



这个页面需要拉到底，等几秒再点I Accept

如果被拦截，则去邮件网关里先点“投递”
在邮箱里看到邮件后，再点相应的链接
(邮件内的网关链接会暂时被屏蔽，是一个假的链接)



2. PyCharm环境搭建

The screenshot shows the JetBrains Account login page. The main heading is "Welcome to JetBrains Account". Below it, the text says: "Link your JetBrains Educational Pack to a new or an existing JetBrains Account. You will need to use this account whenever you want to access JetBrains tools."

There are two main sections for user interaction:

- Sign in with existing account:** This section includes a text input field for the username (containing "Worldfickler"), a password input field (masked with dots), a blue "Sign In" button, and a link for "Forgot password?". A red arrow points from the text "已有账号，登录" (Already have an account, login) to the "Sign In" button.
- Or sign in with:** This section offers social login options with icons for Google, GitHub, and others.
- Not registered yet? Create JetBrains Account:** This section includes a text input field for "Your email address" and a blue "Sign Up" button. A red arrow points from the text "无账号，注册" (No account, register) to the "Sign Up" button.

Additional annotations include:

- A red arrow points from the text "这里的邮箱地址可以填写QQ邮箱或者126等等方便接收的邮箱，因为这个时候你的学生身份已经被官方接受了" (The email address here can be a QQ email or 126 etc. for easy reception, because at this time your student identity has been accepted by the official) to the email input field.
- At the bottom right, there is a black circular button with a white question mark and the word "支持" (Support).

Footer text includes: "By using the JetBrains Account website, you agree to the JetBrains Account Agreement. Review now Remind me later", "Copyright © 2000-2022 JetBrains s.r.o. | Help | Support | JetBrains Privacy Policy | JetBrains Account Agreement | Build #2022.05-2819", and "CSDN @在人间负责".

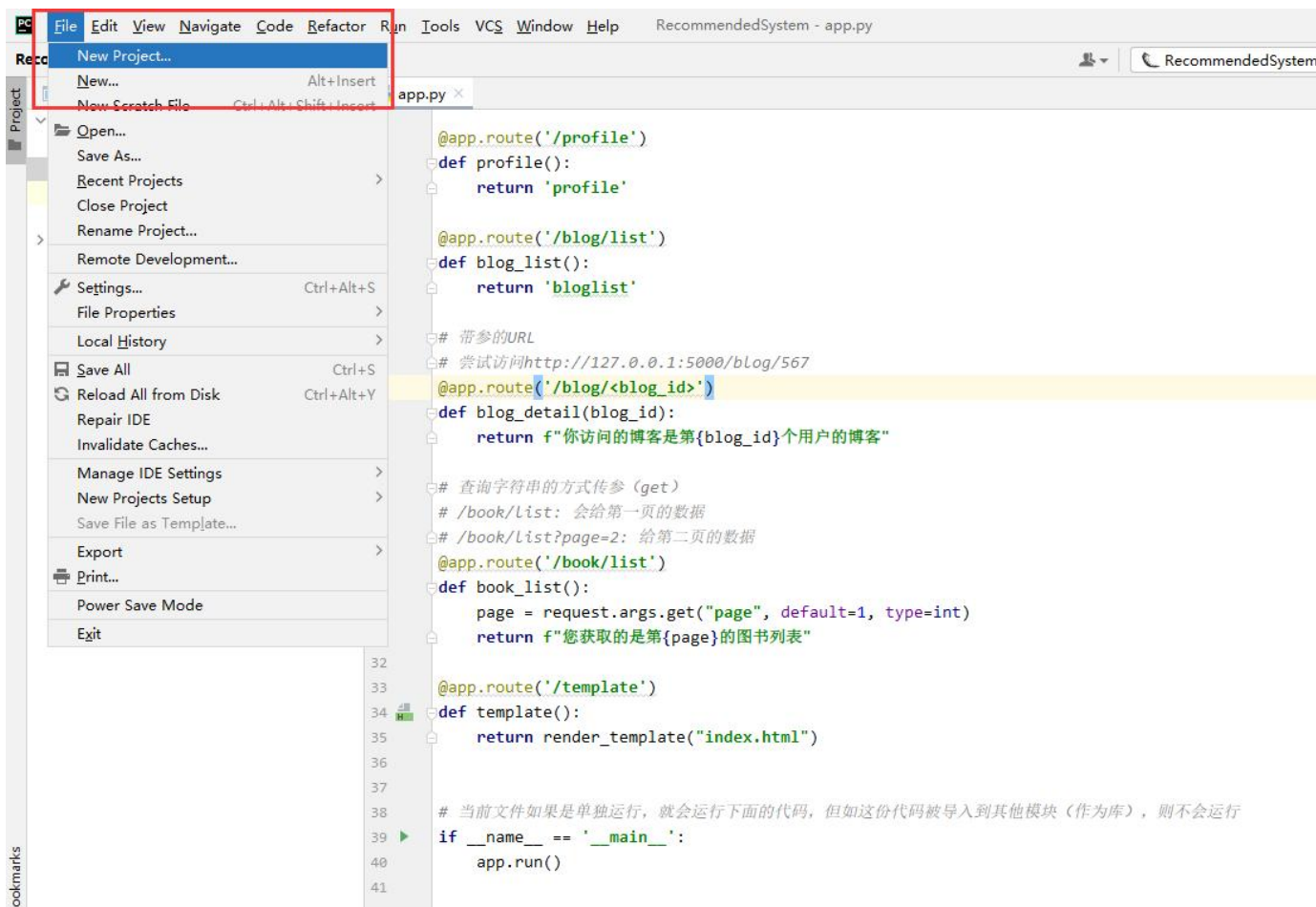


2. PyCharm环境搭建 - 方法一

在pycharm中选择
“New Project”然后选中“Flask”项目（如果你不喜欢使用Flask框架，也可以使用自己熟悉的框架）

如果是使用提供的例子文件，则需要额外安装以下环境：

numpy
pandas
sickit-learn
PyMySQL
tensorflow



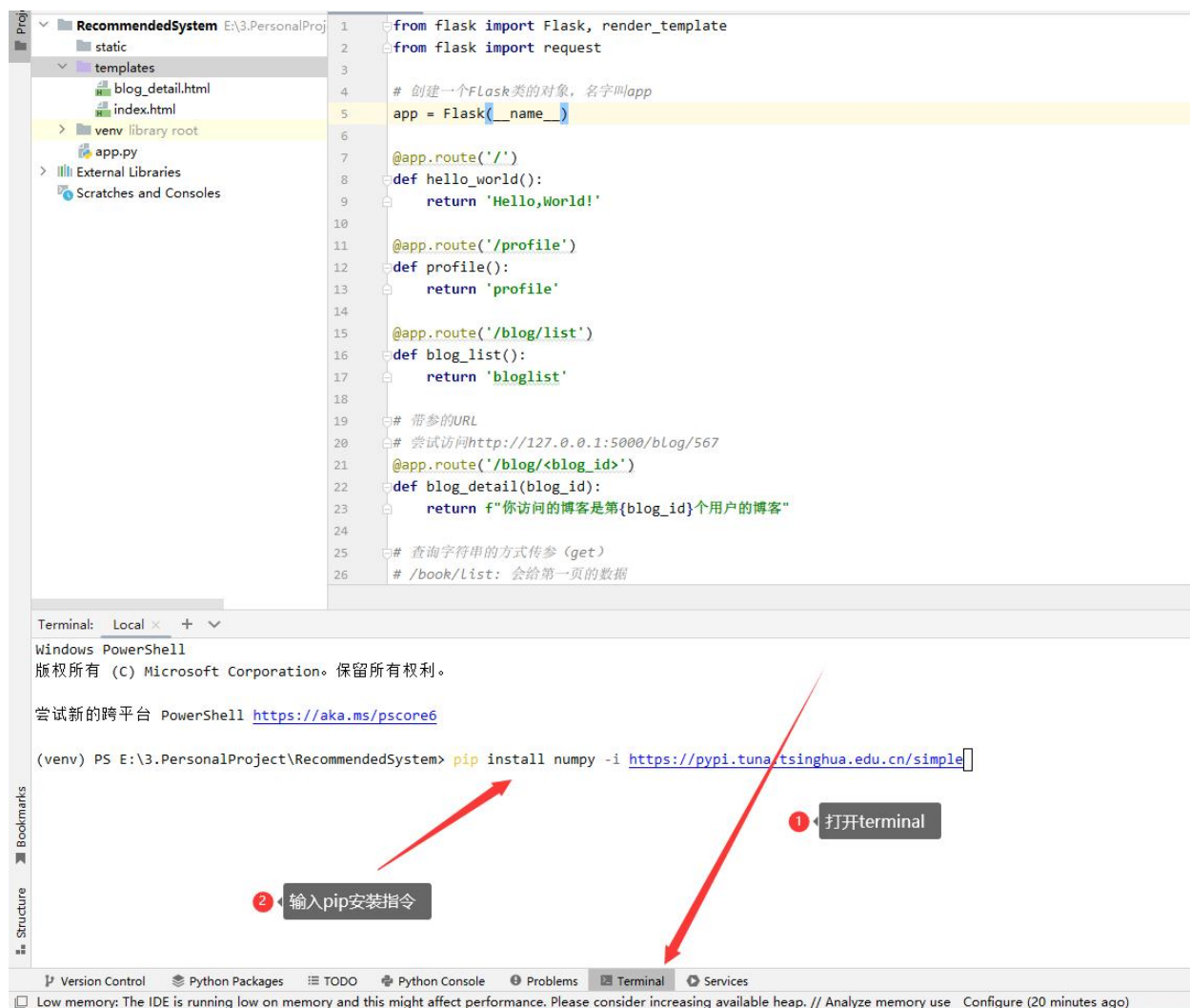


2. PyCharm环境搭建 - 方法一

安装新的
package:

使用File - Setting
- python
interpreter 也可
以更新包, 但是
可能速度会较慢

使用pycharm中的
Terminal输入
指令的形式来安
装包



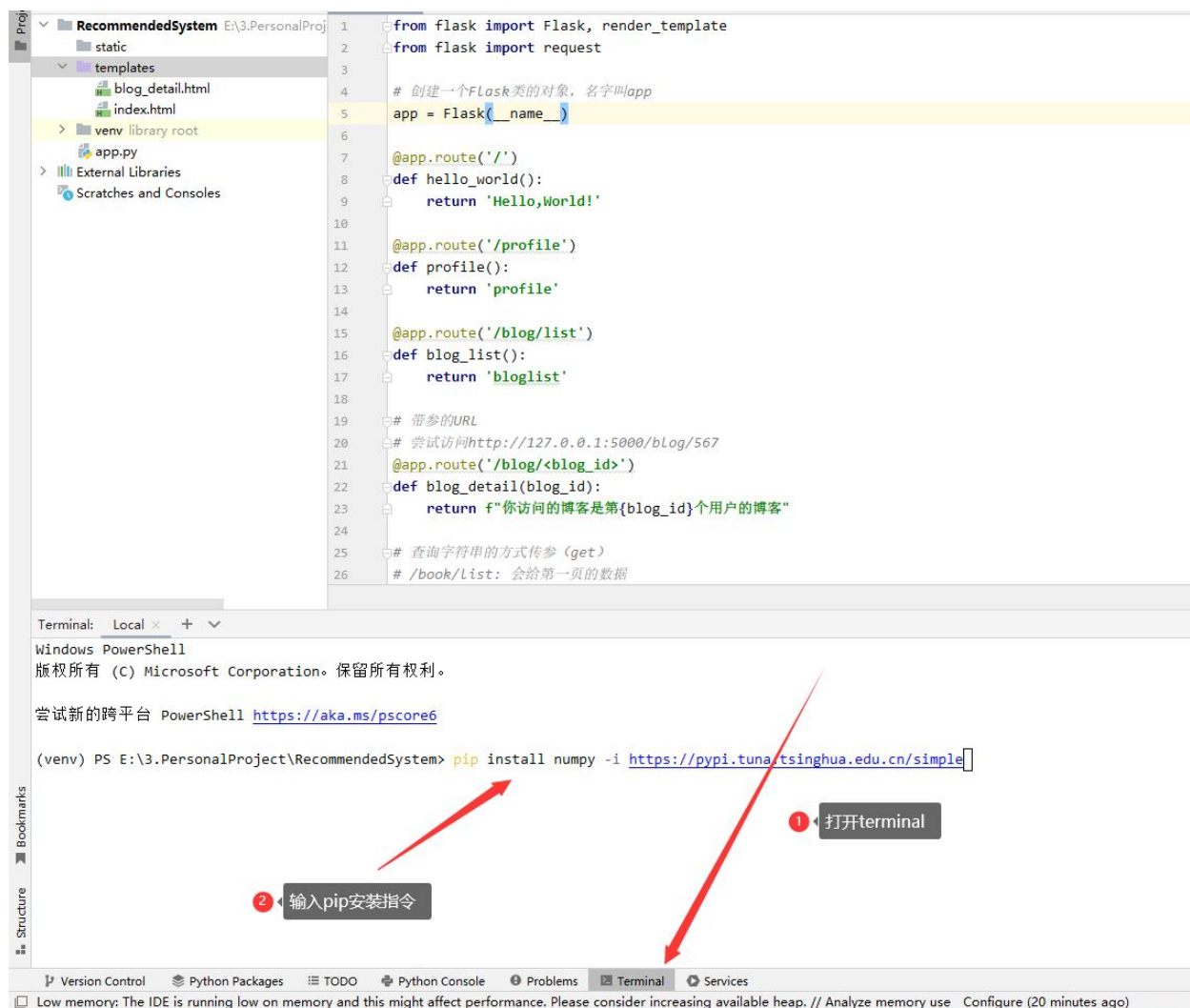


2. PyCharm环境搭建 - 方法一

安装新的
package:

使用File - Setting
- python
interpreter 也可
以更新包, 但是
可能速度会较慢

使用pycharm中的
Terminal输入
指令的形式来安
装包





2. PyCharm环境搭建 - 方法二

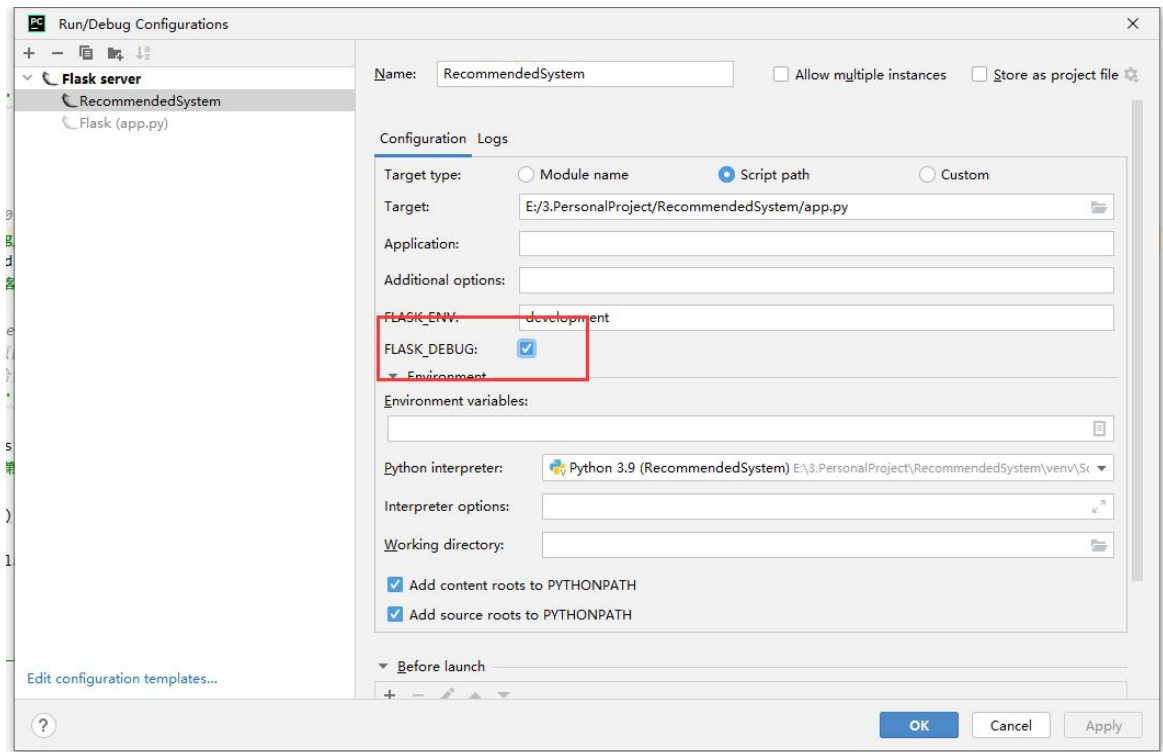
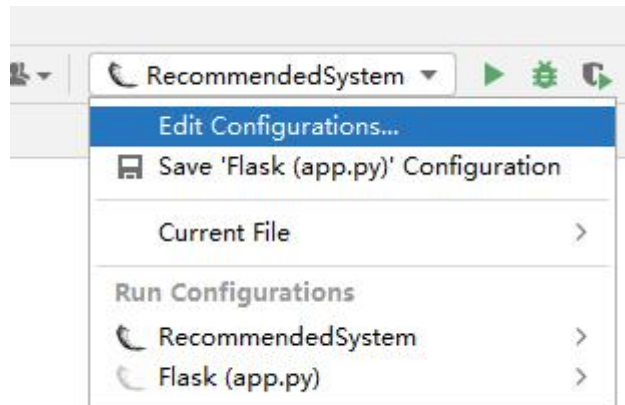
如果新建项目时没有**Flask**选项（一般来说是**community**版本）或者不确认使用包的版本是否符合

则将**requirements.txt**拷贝到项目根目录下，然后在**Pycharm Terminal**（上一页有位置）中执行

```
(venv) pip install -r requirements.txt -i  
https://pypi.tuna.tsinghua.edu.cn/simple
```



2. PyCharm环境搭建



切换debug模式方便修改代码后调试
(修改代码后，项目自动重启，无需手动重启)

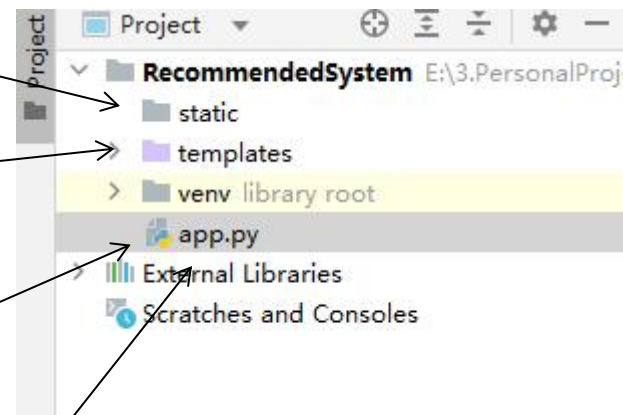


3. 项目结构

static一般来说存放的是静态文件，
图片，视频，**css**，**javascript**等

放**html**模板文件

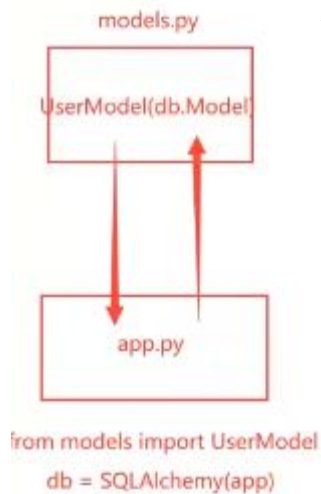
启动的时候启动这个，用
来定义路由，视图函数等



后面可以随意拓展，比如操作数据库
的可以放进一个.py文件，模型可以
放进一个.py文件，由**app.py**调用即
可



3. 项目结构

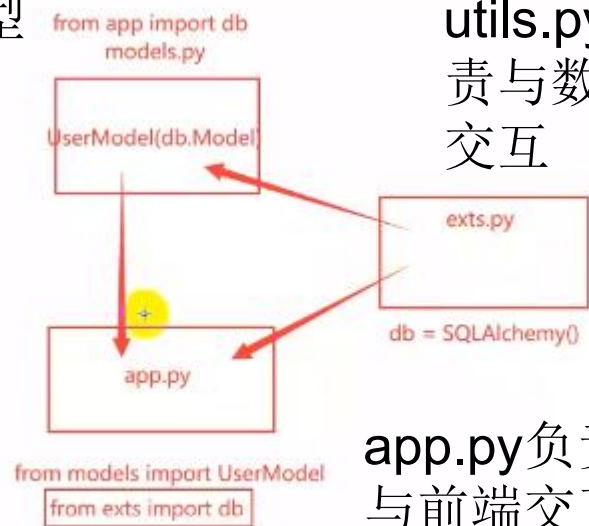


算法和模型

放置了数据库交互逻辑的app.py

× (循环引用)

models.py负责调用算法和模型



有的地方叫utils.py, 负责与数据库交互

app.py负责与前端交互

√

尽量使代码分离，一份代码干一件事



4. 数据库设置

环境搭好后，需要先准备好mysql环境（本机/服务器）

运行以下三个文件：

```
python read_data_into_mysql.py
```

```
python add_password_for_users.py （时间非常长，但是可以跑）
```

```
python add_index_for_tables.py
```

```
(venv) ecs-user@iZ7xv0j1klzkvwwoelrhirZ:~/flaskProject/RecommendedSystem$ python  
add_index_for_tables.py  
数据添加索引完成
```

此时可以去数据库内确认movielens库中的表格
（但是不要用SELECT * 指令）



4. 数据库设置

```
app.py x test.py x read_data_into_mysql.py x models.py x
1 import pymysql
2
3 # 连接到 MySQL 数据库
4 conn = pymysql.connect(host='127.0.0.1',
5                        user='root',
6                        password='123456',
7                        port=3307,
8                        database='movielens',
9                        charset='utf8mb4',
10                       cursorclass=pymysql.cursors.DictCursor)
11
12 # 创建 movies 表
13 with conn.cursor() as cursor:
14     cursor.execute('''CREATE TABLE IF NOT EXISTS movies
15                    (movie_id INT PRIMARY KEY, title TEXT, genres TEXT)''')
16
17 # 创建 users 表
18 with conn.cursor() as cursor:
19     cursor.execute('''CREATE TABLE IF NOT EXISTS users
20                    (user_id INT PRIMARY KEY, gender TEXT, age INT, occupation TEXT, zip_code TEXT)''')
21
22 # 创建 ratings 表
23 with conn.cursor() as cursor:
24     cursor.execute('''CREATE TABLE IF NOT EXISTS ratings
25                    (user_id INT, movie_id INT, rating INT, timestamp INT)''')
26
27 # 读取 movies.dat 文件, 并将数据插入到数据库中
28 with open('./data/movies.dat', encoding='latin1') as file:
29     for line in file:
30         movie_id, title, genres = line.strip().split(':::')
```

端口号, 一般
MySQL的数据库
库为3306

保证当前MySQL数据库
下有该表格

创建表格

使用pymysql将数据导入进数据库



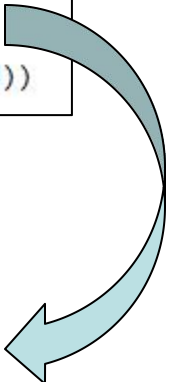
4. 数据库设置

为了实现注册/登录功能，需要在原有数据集user.dat上加上默认密码
这里直接将用户ID作为了密码，通过Werkzeug的generate_password_hash()可直接生成哈希加密后的密码

可执行提供的代码一次性生成到数据库中：add_password_for_users.py
也可以增加一个表，专门管理密码，方便后续推荐算法处理（推荐）

```
for user in tqdm(users, desc="Updating users"):
    user_id = user["user_id"]
    hashed_password = generate_password_hash(str(user_id))

    # 更新用户记录，添加加密后的密码
    cursor.execute("UPDATE users SET password = %s WHERE user_id = %s", (hashed_password, user_id))
```



user_id	gender	age	occupation	zip_code	password
1	F	10		48067	sCrypt:32768:8:1\$hysiJnlDjoJ14EmN\$e958a01b1e
2	M	56	16	70072	sCrypt:32768:8:1\$wbKVuFPaSw7gy1dt\$18e1135fa
3	M	25	15	55117	sCrypt:32768:8:1\$AzorNVj2ApKTzvku\$f6d447e55a
4	M	45	7	02460	sCrypt:32768:8:1\$bUwaXavujYlcp4Bn\$fae98e113c
5	M	25	20	55455	sCrypt:32768:8:1\$Xs56xl2ir6KqJjNs\$1a3a65bcdcb
6	F	50	9	55117	sCrypt:32768:8:1\$gBMhOZcRAOfAkM2\$92b1d5
7	M	35	1	06810	sCrypt:32768:8:1\$CzUVzIYWKmPDcfG7\$33bb932
8	M	25	12	11413	sCrypt:32768:8:1\$CbIT5kAMO54ct8h5\$be2b6816
9	M	25	17	61614	sCrypt:32768:8:1\$h2PslxtTEi50U1zA\$a0d91a7078
10	F	35	1	95370	sCrypt:32768:8:1\$iGFA7ur00up4PCbk\$b2e9702a9
11	F	25	1	04093	sCrypt:32768:8:1\$NP2a8f83GPldVcqu\$cdea7cb72
12	M	25	12	32793	sCrypt:32768:8:1\$hU5Cgr2jeKYIOYnP\$d3af9e26c
13	M	45	1	93304	sCrypt:32768:8:1\$IYCB4RY6lapi5N3\$1f13b5d177



5. 模板继承

- 一个网页中，大部分的网页模块是重复的，比如导航栏，底部相关链接和备案信息等
- 可以将重复的代码作为父文件，创建子文件去继承顶部的父模板，避免每个页面都写的情况。



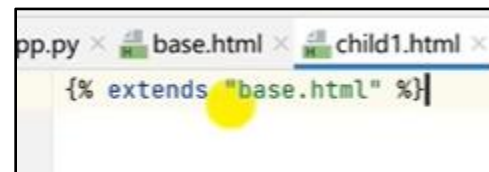
5. 模板继承

最简单的例子



```
pp.py x base.html x control.html x
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>这是父模板</title>
</head>
<body>
  我是父模板的文字
</body>
</html>
```

base.html



```
pp.py x base.html x child1.html x
{% extends "base.html" %}
```

child1.html



访问child1.html



5. 模板继承

在子模版中填充内容

```
app.py x base.html x child1.html x control.html x filter.  
1 <!DOCTYPE html>  
2 <html lang="en">  
3 <head>  
4 <meta charset="UTF-8">  
5 <title>{% block title %}{% endblock %}</title>  
6 </head>  
7 <body>  
8 我是父模板的文字  
9 {% block body %}  
10 {% endblock %}  
11 </body>  
12 </html>
```

base.html

```
app.py x base.html x child1.html x  
{% extends "base.html" %}  
  
{% block title %}  
我是子模板的标题  
{% endblock %}  
  
{% block body %}  
我是子模板的body  
{% endblock %}
```

child1.html



访问child1.html



6. 页面渲染

- 写CSS去渲染页面非常麻烦，有没有一个很大的css文件，里面记录了非常多组件好看的渲染方式，直接调用这个文件来渲染我的html文件？
- 可以使用bootstrap进行渲染

Bootstrap V4文档

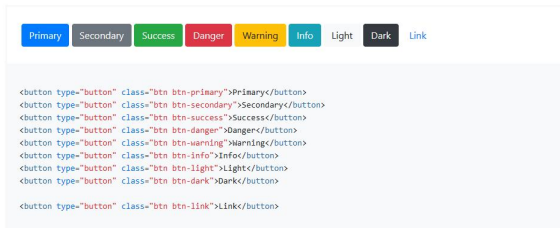
<https://v4.bootcss.com/docs/components/buttons/>

按钮 (Buttons)

Use Bootstrap's custom button styles for actions in forms, dialogs, and more with support for multiple sizes, states, and more.

示例

Bootstrap 内置了几种预定义的按钮样式。每种样式都有自己的语义目的，并添加了一些额外的按钮includes several predefined button styles, each serving its own semantic purpose, with a few extras thrown in for more control.



文档页面

确认密码

立即注册

渲染效果



6. 页面渲染

1. 去官网找到相应组件，查看组件用法

2. 在自己的html中引入css文件，并仿照例子在组建的class=""后面添加相应内容

3. 渲染成功

Bootstrap 内置了几种预定义的按钮样式，每种样式都有自己的语义目的，并添加了一些额外的按钮includes several predefined button styles, each serving its own semantic purpose, with a few extras thrown in for more control.



```
<button type="button" class="btn btn-primary">Primary</button>
<button type="button" class="btn btn-secondary">Secondary</button>
<button type="button" class="btn btn-success">Success</button>
<button type="button" class="btn btn-danger">Danger</button>
<button type="button" class="btn btn-warning">Warning</button>
<button type="button" class="btn btn-info">Info</button>
<button type="button" class="btn btn-light">Light</button>
<button type="button" class="btn btn-dark">Dark</button>

<button type="button" class="btn btn-link">Link</button>
```

```
</div>
{% endfor %}
<button type="submit" class="btn btn-primary btn-block">立即注册</button>
</form>
```

用户名/密码



6. 页面渲染

在flask中加载静态文件时候，请使用url_for函数来实现

（不使用这种方式，直接引用可能会由于相对路径问题，无法加载）

```
</title>
<link rel="stylesheet" href="./static/bootstrap/bootstrap@4.6.1.min.css">
<script src="./static/jquery/jquery.3.6.0.min.js"></script>
{% block header %}
```

直接使用路径，可能会导致无法加载



```
</title>
{# <Link rel="stylesheet" href="./static/bootstrap/bootstrap@4.6.1.min.css">#}
<link rel="stylesheet" href="{{ url_for('static', filename='bootstrap/bootstrap@4.6.1.min.css') }}">
<script src="{{ url_for('static', filename='jquery/jquery.3.6.0.min.js') }}"></script>
{% block header %}
```

使用url_for函数，保证静态文件得以加载



7. app.py

使用钩子函数，在每次请求前执行检查代码，检查session内有没有保存用户ID
如果没有（用户不在登录状态，则设置全局变量user内容为无，否则保存用户的对象）

```
# 钩子函数，在请求前执行
@app.before_request
def before_request():
    # 不必理会解密过程和冲突问题，这些flask都已经帮你做好了
    user_id = session.get("user_id")
    if user_id:
        user = UserModel()
        user.find_by_id(user_id)
        # g, global, 全局变量 from flask import g的g
        # 将用户对象存到全局变量中
        setattr(g, "user", user)
    else:
        # 即便不存在也要设置为None
        setattr(g, "user", None)

@app.context_processor
def context_processor():
    return {"user": g.user}

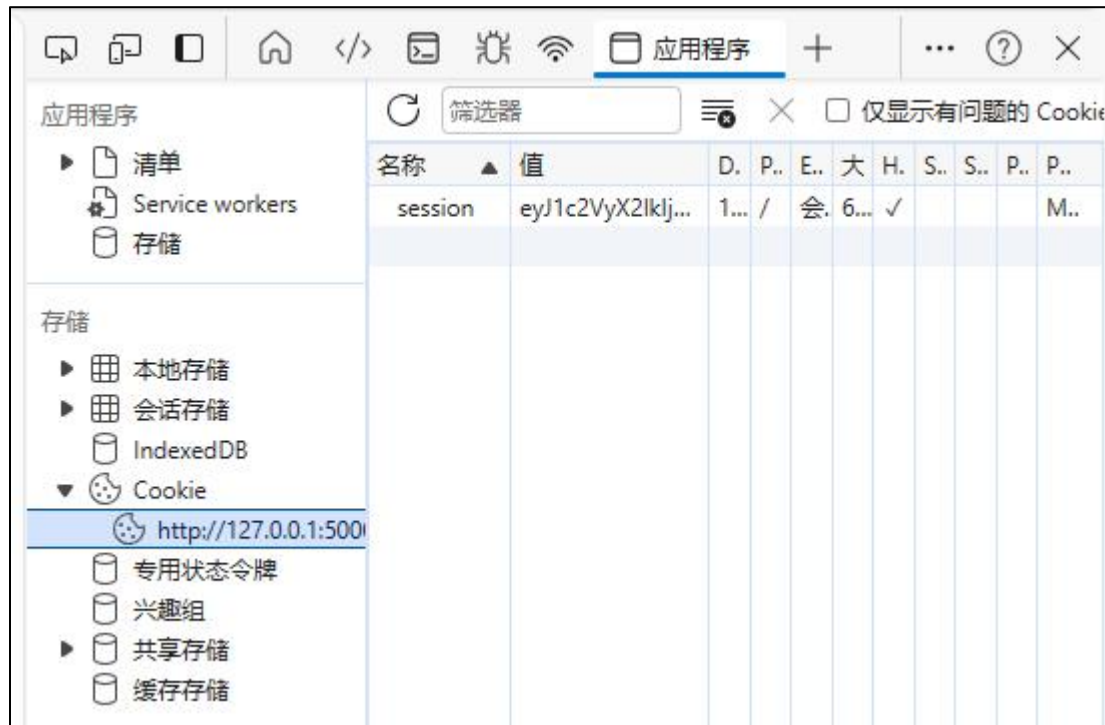
@app.route('/')
def hello_world():
    return render_template("index.html")
```



7. app.py

当用户登录后，执行这一段代码，**flask**把用户**id**存在**session**（会话）里，加密后把**sessionID**存到客户端（浏览器）**cookie**中

```
session['user_id'] = user_model.user_id
```



登录成功后**cookie**中会出现新内容



7. app.py

```
@app.route('/recommend/mostPopular/<user_id>')
def recommend_mostpopular(user_id):
    most_popular_models = MostPopular()
    recommended_movies_detail = most_popular_models.get_most_popular_movies(10)
    print(recommended_movies_detail)
    return render_template("recommended_page.html", user_id = user_id, recommended_movies_detail = recommended_movies_detail, recommendation_method = "热度")

@app.route('/recommend/ItemCF/<user_id>')
def recommend_itemcf(user_id):
    item_cf = ItemCF()
    recommended_movies_detail = item_cf.get_movies_ItemCF(10, user_id)
    print(recommended_movies_detail)
    return render_template("recommended_page.html", user_id = user_id, recommended_movies_detail = recommended_movies_detail, recommendation_method = "物品协同过滤")
```

需要登录访问的界面，从g全局中尝试获取用户对象
如果可以获取到（用户登录了）则根据对象的属性决定推荐内容

```
@app.route('/recommend/ItemCF/')
def recommend_itemcf():
    user = getattr(g, 'user', None)
    if user:
        user_id = user.user_id
        item_cf = ItemCF()
        recommended_movies_detail = item_cf.get_movies_ItemCF(10, user_id)
        print(recommended_movies_detail)
        return render_template("recommended_page.html", user_id=user_id,
                                recommended_movies_detail=recommended_movies_detail,
                                recommendation_method="物品协同过滤")
    else:
        flash('请先登录')
        return redirect(url_for("login"))
```




7. app.py

效果：点击导航栏相应按钮时进行跳转

电影推荐系统

首页

推荐页面

关键字

搜索

登录

注册

可在html内使用url_for实现
相关代码：base.html

url_for('视图函数名字')来进行函数调用

```
{% else %}
<li class="nav-item">
  <a class="nav-link" href="{{ url_for("login") }}">登录</a>
</li>
<li class="nav-item">
  <a class="nav-link" href="{{ url_for("register") }}">注册</a>
</li>
{% endif %}
```

```
@app.route("/login", methods=['GET', 'POST'])
def login():
    return render_template("login.html")
```

```
@app.route("/register", methods=['GET', 'POST'])
def register():
```



7. app.py

相关代码：app.py

```
# 创建一个Flask类的对象，名字叫app
app = Flask(__name__)
app.config['SECRET_KEY'] = os.urandom(24)
```

会话安全性：**Flask** 使用会话来跟踪用户的登录状态和其他信息。如果没有设置密钥，会话数据将以明文形式存储在用户的浏览器中，这会导致安全风险，因为攻击者可以轻松地篡改或窃取会话数据。设置密钥后，会话数据将被加密，提高了会话的安全性。

如果不加，程序在执行**redirect**重定向时会出现如下错误：

```
RecommendedSystem x
rv = self.handle_user_exception(e)
File "E:\3.PersonalProject\RecommendedSystem\venv\lib\site-packages\flask\app.py", line 870, in full_dispatch_request
rv = self.dispatch_request()
File "E:\3.PersonalProject\RecommendedSystem\venv\lib\site-packages\flask\app.py", line 855, in dispatch_request
return self.ensure_sync(self.view_functions[rule.endpoint])(**view_args) # type: ignore[no-any-return]
File "E:\3.PersonalProject\RecommendedSystem\app.py", line 97, in register
flash('Passwords do not match')
File "E:\3.PersonalProject\RecommendedSystem\venv\lib\site-packages\flask\helpers.py", line 323, in flash
session["_flashes"] = flashes
File "E:\3.PersonalProject\RecommendedSystem\venv\lib\site-packages\flask\sessions.py", line 102, in _fail
raise RuntimeError(
RuntimeError: The session is unavailable because no secret key was set. Set the secret_key on the application to something unique and secret.
127.0.0.1 - - [18/May/2024 19:21:03] "GET /register?__debugger__=yes&cmd=resource&f=style.css HTTP/1.1" 304 -
127.0.0.1 - - [18/May/2024 19:21:03] "GET /register?__debugger__=yes&cmd=resource&f=debugger.js HTTP/1.1" 304 -
127.0.0.1 - - [18/May/2024 19:21:03] "GET /register?__debugger__=yes&cmd=resource&f=console.png HTTP/1.1" 304 -
127.0.0.1 - - [18/May/2024 19:21:03] "GET /register?__debugger__=yes&cmd=resource&f=console.png HTTP/1.1" 304 -
```



7. app.py

注册：如果是GET请求，返回注册页面

如果是POST请求，从表单中获取数据，构建用户对象，然后传至数据库

相关代码：app.py

```
@app.route("/register", methods=['GET', 'POST'])
def register():

    if request.method == 'GET':
        return render_template("register.html")

    # POST请求获取用户注册时的用户名密码等，省略了验证步骤
    # 验证可以直接在前端写js，也可以使用后端的验证器，详见：https://www.bilibili.com/video/BV17r4y1y7jJ
    else:
        print("进入POST请求")
        user_id = request.form['user_id']
        gender = request.form['gender']
        age = request.form['age']
        jobs = request.form['jobs']
        postcode = request.form['postcode']
        password = request.form['password']
        password_confirm = request.form['password_confirm']

        print("app.py获取到用户名为：", user_id)
        print("app.py获取到性别为：", gender)
        print("app.py获取到年龄为：", age)
```



7. app.py

登录：如果是GET请求，返回登录页面
如果是POST请求，从表单中获取数据，在数据库中根据ID找到用户相关信息，构建用户对象

相关代码：app.py

```
@app.route("/login", methods=['GET', 'POST'])
def login():
    if request.method == "GET":
        return render_template("login.html")

    else:
        print("进入login的POST请求")
        user_id = request.form['user_id']
        password = request.form['password']
        print("测试：获取到的明文密码为：", password)

        user_model = UserModel()
        result = user_model.find_by_id(user_id)
        print("查询结果为：", result)

        # 查询到有该用户 - 比较密码 - 返回主页
        if result:
            print("用户对象中储存的密码：", user_model.hashed_password)
            print("登录页面获取的密码：", generate_password_hash(password))
            print("是不一致的，需要用check_password_hash来检查，而不是比较")

            if check_password_hash(user_model.hashed_password, password):
                # 需要用到cookie（小曲奇/面包屑）来记录登录状态
                # 因为是屑，不适合存储太多数据，根据教程说一般用来存登录授权信息
                # flask中的session经过加密存储在cookie中

                # 把用户id存在session（会话）里，加密后把sessionID存到客户端（浏览器）cookie中
```



8. 使用ItemCF进行个性化推荐：

相关代码：models.py

用户类包含以下方法：

- `__init__(self)`: 构造方法，初始化用户的成员变量
- `record(self, ...)`: 录入用户成员变量的方法
- `save_to_database(self)`: 将当前用户的属性存放至数据库，注册时调用
- `find_by_id(self, user_id)`: 根据用户ID构造出用户对象，登录时调用，该方法会调用`record()`方法
- `save_rating(self, movie_id, rating)`: 将用户评分数据存放至数据库

较为常规的用户类结构



8. 使用ItemCF进行个性化推荐：

- 什么是个性化推荐？
- 针对不同的用户ID，推荐不同的物品
- 算法如ItemCF， UserCF， 双塔模型等都可以实现个性化的推荐
- 基于热度， 基于性别热度等算法不能实现个性化推荐。

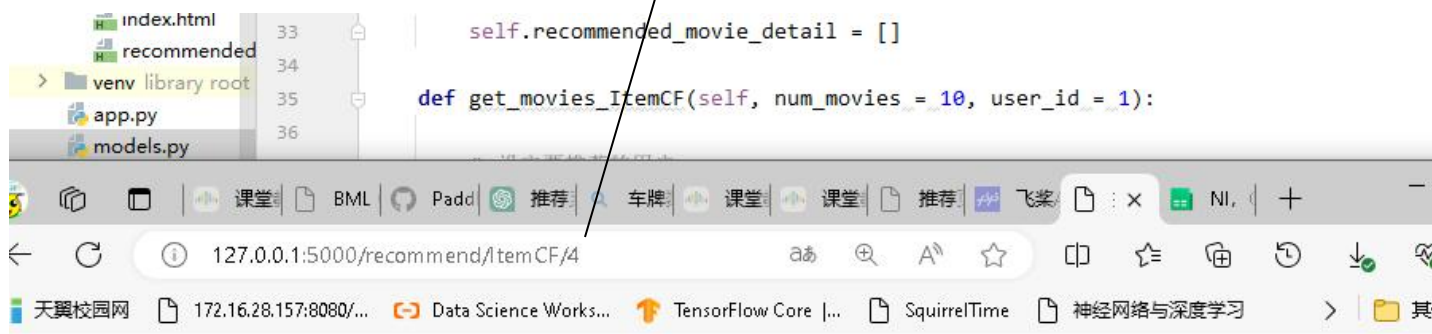


8. 使用ItemCF进行个性化推荐：

- 以ItemCF为例，如何实现个性化推荐？
- 1. 需要获取目标用户的ID

```
@app.route('/recommend/ItemCF/<user_id>')
def recommend_itemcf(user_id):
    item_cf = ItemCF()
    recommended_movies_detail = item_cf.get_movies_ItemCF(10, user_id)
    print(recommended_movies_detail)
    return render_template("recommended_page.html", user_id = user_id,
```

调用ItemCF的时候将
user_id传过去



你好，用户4，你的基于物品协同过滤推荐的列表是：



8. 使用ItemCF进行个性化推荐：

2. 使用学过的算法进行推荐

1. 给定用户ID，通过“用户 $\rightarrow$ 物品”索引，找到用户近期感兴趣的物品列表（last-n）
2. 对于last-n列表中每个物品，通过“物品 $\rightarrow$ 物品”的索引，找到top-k相似物品。
3. 对于取回的相似物品（最多有 $n \cdot k$ 个），用公式预估用户对物品的兴趣分数。
4. 返回分数最高的n个物品，作为推荐结果。

（参考文件：第四周的ItemCF-fix.ipynb）



8. 使用ItemCF进行个性化推荐：

3. 使用for循环控制语句将所有推荐结果反映到前端

```
<body>
  <h1>你好，用户{{ user_id }}, 你的基于{{ recommendation_method }}推荐的列表是：</h1>
  <table>
    <tr>
      <th>电影ID</th>
      <th>标题</th>
      <th>类别</th>
    </tr>
    {% for movie in recommended_movies_detail %}
      <tr>
        <td>{{ movie['movie_id'] }}</td>
        <td>{{ movie['title'] }}</td>
        <td>{{ movie['genres'] }}</td>
      </tr>
    {% endfor %}
  </table>
```



8. 使用ItemCF进行个性化推荐：

推荐页面表格增加一列评分，注意在<form></form>之间要加多一层隐藏数据，表示电影的ID（否则无法知道用户对哪部电影打了分）

```
<tr>
  <td>{{ movie['movie_id'] }}</td>
  <td>{{ movie['title'] }}</td>
  <td>{{ movie['genres'] }}</td>
  <td>
    <form method="POST" action="{{ url_for('submit_rating') }}">
      <input type="hidden" name="movie_id" value="{{ movie['movie_id'] }}">
      <input type="range" name="rating" min="1" max="5" step="1" value="3" id="rating-{{ movie['movie_id'] }}" oninput="updateRatingValue(this)">
      <span id="rating-value-{{ movie['movie_id'] }}">3</span>
      <button type="submit" class="btn btn-primary btn-sm">提交评分</button>
    </form>
  </td>
</tr>
```

后端先验证登录信息，如果通过则调用用户类的save_rating()方法将数据放入数据库

```
@app.route('/submit_rating', methods=['POST'])
def submit_rating():
    user = getattr(g, 'user', None)
    if user:
        movie_id = request.form['movie_id']
        rating = request.form['rating']

        # 将用户打分保存到数据库中
        user.save_rating(movie_id, rating)

        flash('评分提交成功')
        return redirect(url_for('recommend_itemcf'))
    else:
        flash('请先登录')
        return redirect(url_for("login"))
```



8. 使用ItemCF进行个性化推荐：

你好，用户8，你的基于物品协同过滤推荐的列表是：

电影ID	标题	类别		
50	Usual Suspects, The (1995)	Crime Thriller	<input type="range"/>	3 提交评分
318	Shawshank Redemption, The (1994)	Drama	<input type="range"/>	3 提交评分
356	Forrest Gump (1994)	Comedy Romance War	<input type="range"/>	3 提交评分
457	Fugitive, The (1993)	Action Thriller	<input type="range"/>	3 提交评分
590	Dances with Wolves (1990)	Adventure Drama Western	<input type="range"/>	3 提交评分
593	Silence of the Lambs, The (1991)	Drama Thriller	<input type="range"/>	3 提交评分
1196	Star Wars: Episode V - The Empire Strikes Back (1980)	Action Adventure Drama Sci-Fi War	<input type="range"/>	3 提交评分
1610	Hunt for Red October, The (1990)	Action Thriller	<input type="range"/>	3 提交评分
1617	L.A. Confidential (1997)	Crime Film-Noir Mystery Thriller	<input type="range"/>	3 提交评分
2762	Sixth Sense, The (1999)	Thriller	<input type="range"/>	3 提交评分

至此，如果算法使用恰当，那么在用户评分之后，所产生的推荐应该都为实时的



9. 如何获得参考项目

- 充分利用好 <https://github.com/>
- 与同学分享好的社区/记录好的免费素材库
- 当今搜索出来的东西/视频有大概率是付费的
- 不推荐去购买付费项目，很多付费项目也是从免费网站上扒下来改的（我的一个25届毕业生使用了600块买了毕设，买的还不如免费的毕业项目）
- 很多人都做过推荐系统的毕业设计，github肯定有免费开源的各种项目



9. 如何获得参考项目

35 results (230 ms)

Sort by: Best match

Filter by

- <> Code 2.5k
- Repositories 35
- Issues 208
- Pull requests 6
- Discussions 5
- Users 0
- More

Languages

- Python
- HTML
- JavaScript
- C#
- Dart
- Java
- Jupyter Notebook
- Kotlin
- More languages...

Advanced

- Owner
- Size
- Number of followers
- Number of forks

1. **lsq960124/Flask-BookRecommend-Mysql**
使用Flask, mysql构建的一个基于书籍, 基于协同过滤算法, 基于slope one的图书推荐系统
Python · ☆ 267 · Updated on 2023年9月27日

2. **xiefan-guo/studentTrainPlan**
Python+Flask+MySQL实现的学生培养计划管理系统, 项目包括课程推荐、课程评分、交流论坛和模拟选课模块。
Python · ☆ 165 · Updated on 2019年6月3日

3. **caodaqian/Music-Recommend** (Public archive)
音乐推荐系统, python编写, 涉及flask框架, scrapy爬虫, MySQL数据库, selenium, chrome driver, 使用surprise库协同过滤算法
Python · ☆ 75 · Updated on 2019年5月22日

4. **Microstrong0305/Chinese-Spark-movie-lens**
基于Spark、Python Flask和MovieLens dataset的在线电影推荐系统
Jupyter Notebook · ☆ 21 · Updated on 2019年12月20日

5. **dreamcity/Spark_Movie_recsys**
在Spark环境下, 利用Flask框架, 采用MongoDB设计的一个在线电影推荐系统的演示demo
Python · ☆ 21 · Updated on 2016年5月4日

6. **tedqin/RCMSYS**
上创项目-flask-python-基于协同过滤的图书推荐系统
HTML · ☆ 26 · Updated on 2019年7月29日

Flask
Flask is a web framework for Python based on the Werkzeug toolkit.
flask.pocoo.org
[Wikipedia](https://en.wikipedia.org/wiki/Flask_(web_framework))
[pallets/flask](https://pallets.org/flask)
View topic

Sponsor open source projects you depend on
Contributors are working behind the scenes to make open source better for everyone—give them the help and recognition they deserve.
[Explore sponsorable projects](#)

How can we improve search? [Give feedback](#)

ProTip! Press the [/](#) key to activate the search input again and adjust your query.

github上有非常多免费的开源项目
(可以找到数据集等线索)



Thanks !